
Conception Orientée Objet

Enseignante: Salma BRADAI

Niveau Auditoire: CPI2

Année universitaire: 2025-2026



Plan de cours

1- Introduction à la COO & à UML

2- Diagramme des Cas d'Utilisation

3- Diagramme de Classes & Diagramme d'Objets

4- Diagrammes Comportementaux: Séquence & Activité

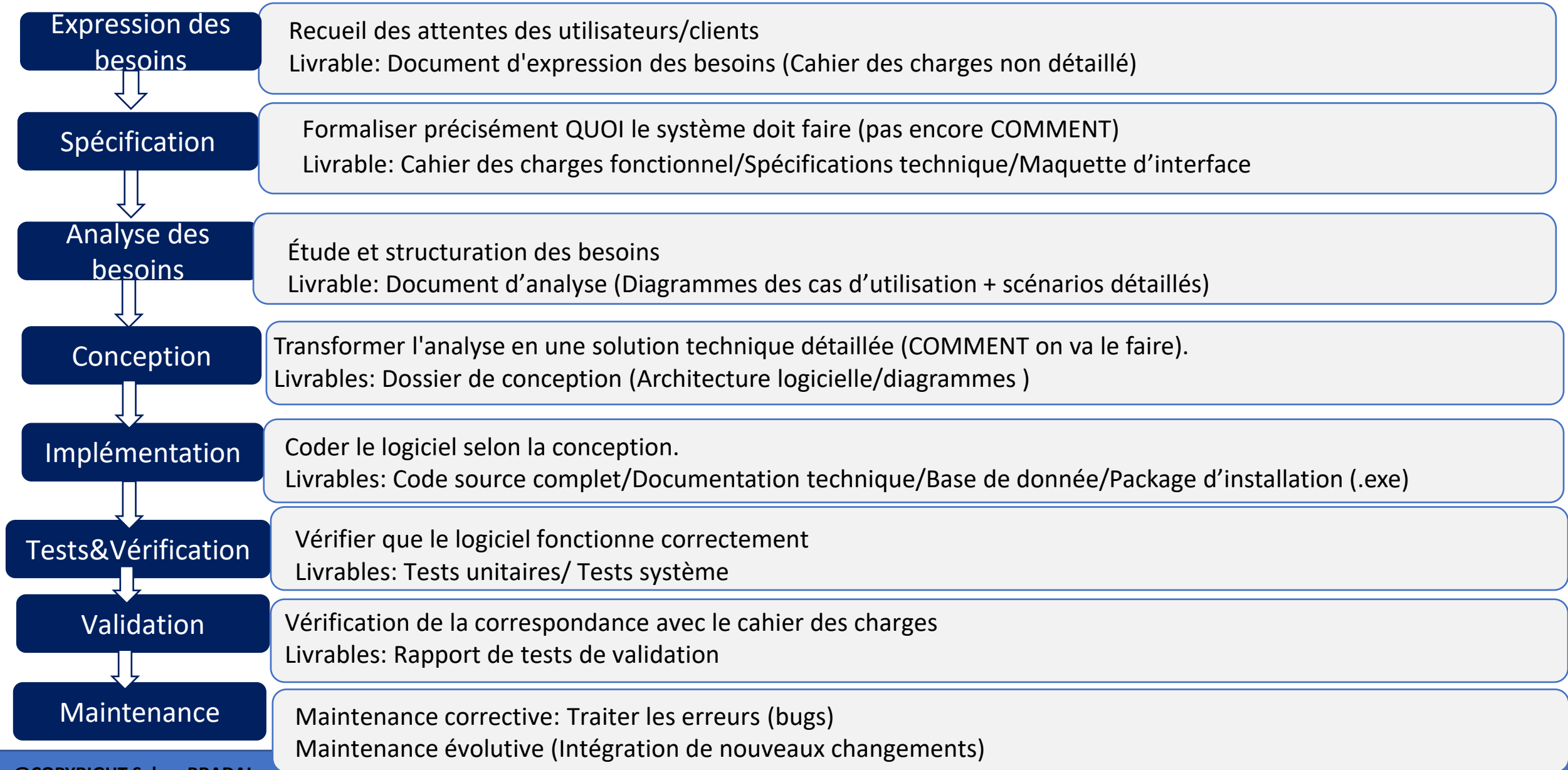
5- Etude de cas

Objectif de cours

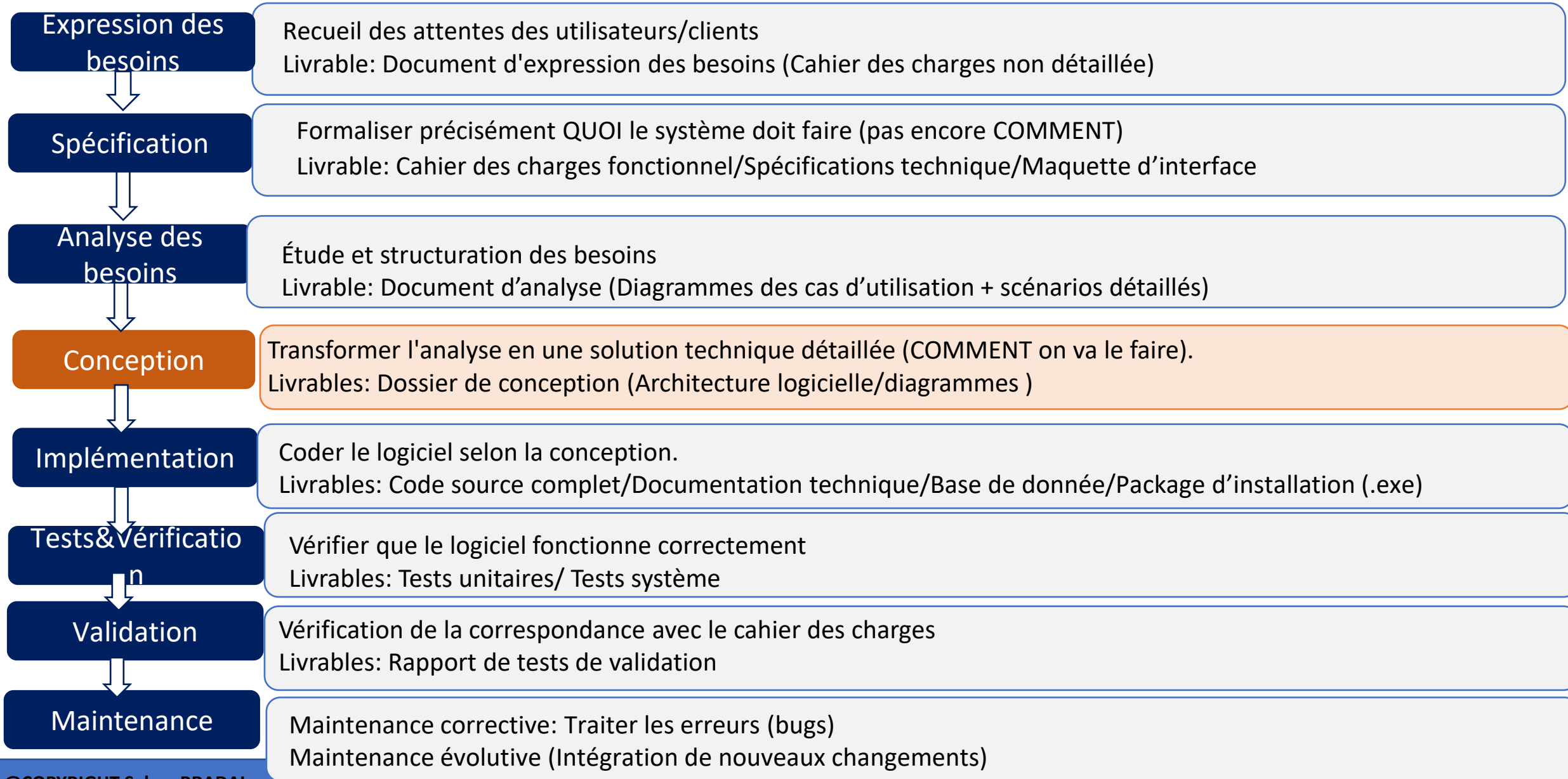
- ❖ Transformer des besoins utilisateurs en solutions logicielles robustes, maintenables et évolutives avant de passer au codage.
 - Maîtriser la pensée orientée objet
 - Acquérir une méthodologie de conception rigoureuse
 - Maîtriser UML comme langage de communication
 - Comprendre comment les diagrammes se traduisent en code

Introduction à la COO & à UML

Cycle de vie d'un logiciel



Cycle de vie d'un logiciel



Conception

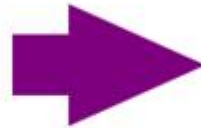
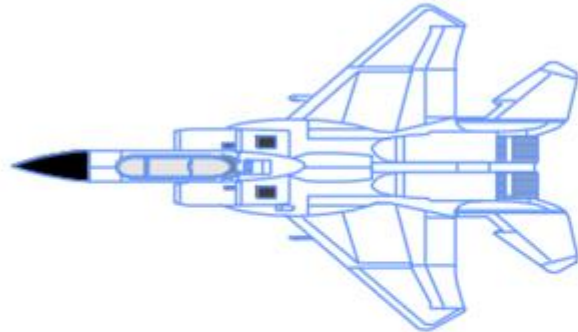
———— Définitions de base

Créer et présenter les diverses fonctionnalités du programme/ logiciel à travers des **modèles** permettant de réaliser rapidement un ou plusieurs programmes réalisant ces fonctions.

Méthode de Conception Orientée Objet

Un modèle

Une simplification de la réalité complexe



Méthode de Conception Orientée Objet

Un modèle

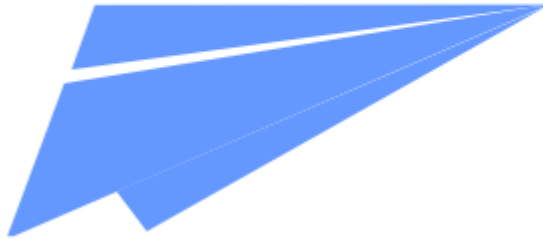
- Quatre objectifs à la modélisation
 - Aider à **visualiser** le système
 - **Spécifier** la structure et le comportement
 - **Servir de plan** pour la construction effective
 - Permettre de **documenter** les choix
- Quatre avantages
 - Abstraction : diviser pour régner ➔ gérer la complexité
 - Compréhension : mises au point avec le client
 - L'énergie déployée pour modéliser révèle les difficultés
 - Les erreurs sur les modèles coûtent bien moins **cher**

Méthode de Conception Orientée Objet

Un modèle

moins important

Plus important



Avion papier



Avion militaire

Le développement logiciel AUSSI nécessite des modèles bien pensés !

Plus un système est important et complexe, plus il nécessite des modèles précis.

Conception

Définitions de base

Créer et présenter les diverses fonctionnalités du programme/ logiciel à travers des **modèles** permettant de réaliser rapidement un ou plusieurs programmes réalisant ces fonctions.

- Elle se base sur la **Modularité**.

Conception

Modularité

La **modularité** est un principe de conception qui consiste à **décomposer un logiciel en modules distincts**, chacun ayant :

- une **responsabilité bien définie**,
- une **interface claire**,
- et une **relative indépendance** par rapport aux autres modules.



👉 Un **module** est une unité logicielle (classe, package, composant, service...) qui peut être comprise, développée et modifiée sans connaître tout le système.

Conception

Définitions de base

Créer et présenter les diverses fonctionnalités du programme/ logiciel à travers des **modèles** permettant de réaliser rapidement un ou plusieurs programmes réalisant ces fonctions.

- Elle se base sur la **Modularité**.
- Une bonne Conception participe largement à la production d'un logiciel qui répond aux **facteurs de qualité**.

Conception

Facteurs de qualité

- **Un module très cohésif :**

a une responsabilité claire et unique et contient des éléments fortement liés à un même objectif.

- **Un module peu cohésif :**

fait plusieurs choses différentes et mélange des responsabilités sans lien direct

✓ *On cherche une forte cohésion*

Cohésion



- **Couplage fort :**
un module dépend fortement des autres. Un changement dans un module entraîne des changements ailleurs.

- **Couplage faible :**
les modules sont relativement indépendants les changements sont localisés

✓ *On cherche un faible couplage*

Couplage



- **Compréhensibilité dépend de:**

- Sa cohésion: Plus cohésif plus compréhensible

- L'appellation: Utilisation de noms significatifs

- La documentation : Lien entre le monde réel et le composant

- La complexité: moins complexe plus compréhensible

Compréhensibilité



- **La capacité du logiciel à:**

- évoluer
- être modifié sans casser l'existant

➔ Couplage faible

➔ Documentation claire facilitant la maintenance

Adaptabilité



Méthodes de Conception

Fonctionnelles

- Décomposant le système en fonctions ou traitements
- « Quelles fonctions le système doit-il assurer? »

✓ Simple à comprendre

✗ Difficile à faire évoluer

Systémiques

- Considère le logiciel comme un système globale, composé de sous systèmes en interactions avec son environnement .
- « Comment le système interagit avec son environnement ? »

✓ Bonne compréhension du système dans son ensemble

✗ Moins précise sur les détails d'implémentation

Orientée Objet

- le logiciel est considéré comme une collection d'objets dissociés définis par des propriétés.
- « Quels sont les objets du système et comment interagissent-ils ? »

✓ Forte modularité et cohésion et faible couplage

Méthode de Conception Orientée Objet

Dans une approche Orientée Objet (OO), le logiciel est considéré comme une collection d'objets dissociés définis par des propriétés.

- Une propriété est soit un attribut de l'objet ou une opération sur l'objet.
- Un objet comprend donc à la fois une structure de données et une collection d'opérations (son comportement).

Contrairement aux méthodes fonctionnelles et systémiques, les méthodes orientées objets sont ascendantes (bottom-up).

Méthode de Conception Orientée Objet

- Guide la conception par:
 - Un ensemble de concepts
 - Abstraction, Modularité, Encapsulation, Polymorphisme
- Des langages et outils qui supportent ces concepts
 - Héritage (Simple, Multiple)
 - Typage (Fort, Faible)
- Avantages
 - Reflète plus finement les objets du monde réel
 - Du code : facile à maintenir
 - Plus stable: Isolation des changements
 - Réutilisation des composantes
 - Faciliter le prototypage

Méthode de Conception Orientée Objet

Les Objets ...

Entité cohérente rassemblant des données et du code travaillant sur ces données.
L'objet informatique est une projection de l'objet du monde réel:
Il peut représenter une entité physique, conceptuelle ou logicielle du monde réel.

- Caractérisé par :
 - son comportement : que peut-on faire avec cet objet?
→ Méthodes
 - son état : comment réagit l'objet quand on applique ces méthodes?
→ Attributs (Champs)
 - son identité : comment distinguer les objets qui ont le même état et le même comportement?
→ Identifiant

Méthode de Conception Orientée Objet

Exemple

- Système de Gestion d'un Lycée

Objets

Personnes

- Etudiant, enseignant, secrétaire

Diplôme

Matière

Classe

...

Fonctions

- Calculer la moyenne
- Calculer les taux d'encadrement
- Calculer le nombre de redoublants
- Calculer le taux de réussite au baccalauréat

.....

Unified Modeling Language (UML)

- Langage visuel unifié
 - Tout le monde doit parler le même langage
 - UML n'est pas une méthode de conception mais un langage. Il permet de représenter des modèles, mais il ne définit pas le processus d'élaboration de ces modèles.
- Langage pour spécifier
 - Supposé précis et non ambigu
- C'est un **support de communication** performant, il facilite la compréhension de représentations abstraites complexes.
- Des liens vers +s langages de prog.
 - Java, C++, VB
 - Génération de code et *reverse engineering*.

Unified Modeling Language (UML)

Historique

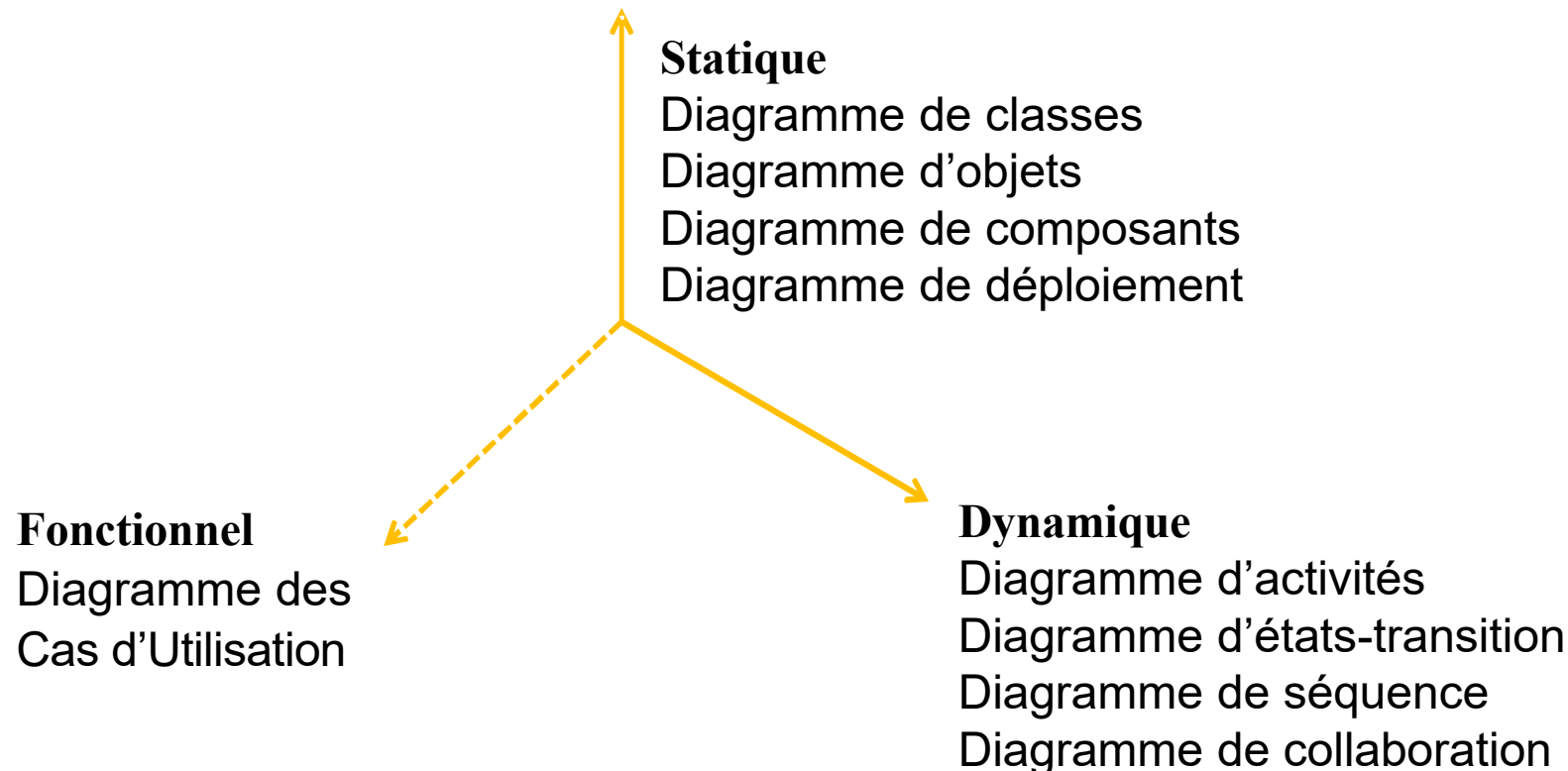
- **Années 80:**
 - Méthodes pour organiser la programmation fonctionnelle (Merise)
 - Séparation des données et des traitements
- **Début des années 90:**
 - Apparition de la programmation orientée objet: nécessite d'une méthodologie adaptée
 - Apparition de plus de 50 méthodes entre 1990 et 1995
- **1994: Consensus sur 3 méthodes**
 - **OMT** de James Rumbaugh : représentation graphique des aspects statiques, dynamiques et fonctionnels d'un système
 - **OOD** de Grady Booch: concept de package
 - **OOSE** d' Ivar Jacobson: description des besoins de l'utilisateur
- **1997: Définition de la norme UML comme standard de modélisation des systèmes d'information objet par l'OMG (Object Management Group)**
 - UML est employé dans l'ensemble des secteurs du développement informatique: Systèmes d'information/ Télécommunication, défense/ Transport, aéronautique, aérospatial/ Domaines scientifiques

Mais pas seulement : on peut modéliser des comportement mécaniques, humain, etc.

Unified Modeling Language (UML)

Axes de modélisation

- UML modélise le système suivant trois axes : le premier décrit les **fonctionnalités** du système, le second concerne la **structure statique** du système et le troisième traite son **aspect dynamique**. Ces trois axes sont nécessaires et complémentaires.



Unified Modeling Language (UML)

Axes de modélisation

- L'axe **fonctionnel** s'appuie exclusivement sur le diagramme de cas d'utilisation. Il est utilisé dans l'activité de spécification des besoins.
- L'axe **statique** ou **structurel** s'appuie sur les diagrammes :
 - Le diagramme de **classes** : est le point central dans un développement orienté objet. En analyse, il a pour objet de décrire la structure des entités manipulées par les utilisateurs.
 - Le diagramme **d'objets** : représente une instance spécifique d'un diagramme de classes à un moment précis
 - Le diagramme de **composants** : représente le système sous forme de composants logiciels (les fichiers source, les packages de code, les bibliothèques,...) réutilisables mettant en évidence leurs relations de dépendance.
 - Le diagramme de **déploiement** : se rapproche encore plus de la réalité physique, puisqu'il identifie les éléments matériels (PC, Modem, Station de travail, Serveur, etc.), leur disposition physique (connexions) et la disposition des composants logiciels sur ces éléments matériels.

Unified Modeling Language (UML)

Axes de modélisation

- L'axe **dynamique** ou **comportemental** s'appuie sur les diagrammes ci-après :
 - Le diagramme **d'activités** : représente les règles d'enchaînement des activités dans le système.
 - Le diagramme **d'états–transition** représente la transition des états (cycle de vie) par lesquels passe un objet de la création à la suppression. Ce diagramme permet de compléter la connaissance des classes.

Unified Modeling Language (UML)

Axes de modélisation

- Les diagrammes **de séquence et de collaboration** : sont tous deux des diagrammes d'interaction UML. Ils représentent les échanges de messages entre objets, dans le cadre d'un fonctionnement particulier du système.
 - Les diagrammes de séquence servent d'abord à développer, en analyse, les scénarios d'utilisation du système.

Unified Modeling Language (UML)

Besoin d'une démarche

- UML est un langage, mais non une méthode. Cependant, ses auteurs préconisent d'utiliser une démarche :
 - **Itérative et incrémentale:** pour modéliser (comprendre et représenter) un système complexe, il vaut mieux l'effectuer en plusieurs itérations, en raffinant à chaque fois la solution obtenue.
 - **Guidée par les besoins des utilisateurs du système:** avec UML, ce sont les utilisateurs qui guident la définition des modèles. Le but du système à modéliser est de répondre aux besoins de ses utilisateurs.
 - **Centrée sur l'architecture logicielle:** Une architecture adaptée est la clé de succès d'un développement. Elle décrit des choix stratégiques qui déterminent en grande partie les qualités du logiciel.

C'est la fin!